

Ficha Técnica — Score Transparente & Plano de Evolução de Crédito

Especificação técnica do sistema: stack, arquitetura, schema do banco, endpoints, variáveis de ambiente e lacunas conhecidas.

Stack

Camada	Tecnologia
Backend	Python 3.11+, FastAPI, Pydantic v2
Banco de dados	SQLite (via sqlite3 da stdlib, sem ORM)
Autenticação	JWT (HS256) via python-jose
Frontend	HTML/CSS/JS vanilla, sem framework, sem build step
Testes	unittest (stdlib), pytest pros testes de API
Hospedagem alvo	Render (backend como Web Service, frontend como Static Site)

Sem Node, sem bundler, sem TypeScript. Decisão deliberada pra reduzir superfície de dependência e manter o frontend abrível por duplo-clique.

Arquitetura

```
backend/app/
├─ core/           # lógica de negócio pura – zero I/O, zero
dependência externa
├─ integrations/  # clientes HTTP reais (Pluggy, Asaas,
SOA/Serasa, SendGrid)
├─ main.py        # todos os endpoints FastAPI
├─ auth.py        # JWT: criação/validação de token, dependency
de auth
├─ schemas.py     # modelos Pydantic (request/response)
└─ database.py    # schema SQL, conexões, queries auxiliares
```

Princípio arquitetural central: tudo que é regra de negócio pura (cálculo de score, máquina de estados de negociação, elegibilidade de marketplace, validação de CPF, decisão de cache) vive em `app/core/`, sem importar `httpx`, `fastapi` nem nada que precise de rede. Isso é o que torna 223 dos ~230 testes executáveis sem nenhuma dependência instalada — só `stdlib`. `app/integrations/` isola tudo que precisa de rede (e por isso não pôde ser testado no ambiente onde este projeto foi gerado).

Não há camada de repositório/ORM — `database.py` expõe funções puras que recebem uma conexão `sqlite3` já aberta (`obter_conexao()` é um context manager). Todas as queries usam parâmetros (?), nunca f-string — decisão deliberada contra SQL injection, não descuido.

Módulos de app/core/ — mapa de responsabilidade

Módulo	Responsabilidade	Testes
score_engine.py	Cálculo de score, impacto de ações, montagem do plano de ação	test_score_engine.py
negociacao.py	Máquina de estados (StatusNegociacao), cálculo de comissão	test_negociacao.py
marketplace.py	Elegibilidade de crédito pré-qualificado (percentual, nunca boolean “aprovado”)	test_marketplace.py
security.py	Hash PBKDF2, validação CPF/CNPJ, hash determinístico (cache), mascaramento	test_security.py
chat_suporte.py	Motor de intenção por palavra-chave (sem LLM externo)	test_chat_suporte.py
account_lockout.py	Regra de bloqueio após N tentativas de senha	test_account_lockout.py
password_reset.py	Token de reset (hash SHA-256, uso único, expiração)	test_password_reset.py
rate_limiter.py / rate_limiter_redis.py	Sliding window em memória / contador Redis (não testado com Redis real)	test_rate_limiter.py
billing.py	Preços, interpretação de webhook Asaas, validação de token de webhook	test_billing.py
cache_score.py	Validade de cache de consulta ao birô (30 dias)	test_cache_score.py
serasa_adapter.py	Parsing da resposta CredNet/Excluir → tipos internos	test_serasa_adapter.py
email_templates.py	Montagem de assunto/corpo de e-mail (sem envio)	test_email_templates.py

Schema do banco (SQLite)

```

    usuarios (id, documento_hash, email, senha_hash, criado_em,
              is_admin, tentativas_login_falhas, bloqueado_ate)
    negociacoes (id, usuario_id, credor, valor_original,
valor_com_desconto,
                  status, criado_em, asaas_payment_id, comissao_paga)
    negociacao_eventos (id, negociacao_id, status, detalhe, timestamp)
    assinaturas (id, usuario_id, plano, asaas_customer_id,
                  asaas_subscription_id, status, criado_em)
    clientes_asaas (usuario_id, asaas_customer_id, criado_em)
    creditos_consulta_avulsa (id, usuario_id, asaas_payment_id, status,
                              criado_em, usado_em)
    cache_consulta_serasa (documento_hash, usuario_id, resposta_json,
consultado_em)
    tokens_reset_senha (token_hash, usuario_id, criado_em, expira_em,
usado)
    log_auditoria (id, usuario_id, timestamp, score_anterior,
score_novo, payload_json)

```

documento_hash em usuarios usa PBKDF2 (não determinístico, não serve pra busca). documento_hash em cache_consulta_serasa usa SHA-256 puro determinístico (hash_deterministico_documento()) — as duas funções são propositalmente diferentes, não confundir.

Endpoints — visão geral (tags no OpenAPI)

Tag	Endpoints principais
auth	/auth/cadastro, /auth/login, /auth/esqueci-senha, /auth/redefinir-senha
score	/score/diagnostico (grátis, manual), /score/diagnostico- serasa (pago, real, gated por assinatura/crédito)
negociacao	CRUD de negociação + /negociacao/{id}/proposta, /transicao, /historico, /cobrar- comissao
marketplace	/marketplace/ofertas
pagamentos	/pagamentos/assinatura, /pagamentos/consulta-avulsa, /pagamentos/webhook
suporte	/suporte/chat
admin	/admin/usuarios, /admin/auditoria, /admin/negociacoes, /admin/gastos-serasa, /admin/consulta-manual-serasa, ações de reset/desbloqueio
infra	/health

Documentação interativa completa: GET /docs (Swagger UI, gerado automaticamente pelo FastAPI).

Variáveis de ambiente (obrigatórias vs. opcionais)

Obrigatórias pra subir a aplicação

```
JWT_SECRET_KEY          # sem isso, app.main falha no import (fail-
fast deliberado)

# Obrigatórias pro banco sobreviver a redeploy no Render
DATABASE_PATH           # ex: /var/data/dados_app.db, exige disco
persistente anexado

# Admin (só necessárias na primeira subida – remover depois)
ADMIN_BOOTSTRAP_EMAIL
ADMIN_BOOTSTRAP_SENHA

# Integrações (cada uma opcional – endpoint correspondente retorna
# 503 educadamente se ausente, não derruba o resto da app)
SOA_EMAIL / SOA_SENHA / SOA_AMBIENTE      # Serasa via SOA Web
Services
ASAAS_API_KEY / ASAAS_SANDBOX / ASAAS_WEBHOOK_TOKEN
SENDGRID_API_KEY / SENDGRID_FROM_EMAIL
PLUGGY_CLIENT_ID / PLUGGY_CLIENT_SECRET
REDIS_URL                               # opcional, rate
limiter distribuído

# Infra
APP_ENV                     # "dev" expõe token de reset de senha na
resposta da API – nunca em produção
CORS_ALLOWED_ORIGINS       # lista separada por vírgula
```

Rodando os testes

```
# Lógica pura – zero dependência instalada, roda em qualquer Python
3.11+
python -m unittest discover -s tests -v

# Suite completa, incluindo API (precisa de pip install -r
requirements.txt)
pytest tests/ -v
```

tests/test_api.py usa TestClient do FastAPI e sobe um SQLite temporário isolado por teste (fixture banco_limpo). Não requer nenhuma credencial real — as integrações externas não são chamadas nos testes de API (os endpoints que dependem delas retornariam 503 sem as variáveis de ambiente, o que os testes atuais não cobrem — ver “Lacunas conhecidas” abaixo).

Integrações externas — peculiaridades que não são óbvias

- **SOA Web Services (Serasa):** autenticação é email/senha **no corpo de cada requisição**, não header nem OAuth. URLs de homologação e produção confirmadas em serasa_client.py — produção ainda não testada com tráfego real no momento da entrega.
 - **Asaas:** webhooks não têm HMAC (confirmado na doc oficial) — a autenticação é um token estático configurado no painel deles, comparado em tempo constante (token_webhook_valido).
 - **Pluggy:** fluxo padrão de Open Finance (API Key → Connect Token) — implementado mas **não integrado a nenhum endpoint ainda**, só o cliente e o adapter existem (pluggy_client.py, pluggy_adapter.py).
-

Lacunas conhecidas (não escondidas — documentadas pra você priorizar)

1. Nenhuma integração externa (SOA, Asaas, Pluggy) foi testada com tráfego de rede real — só a lógica de parsing/decisão, com dado real de resposta quando disponível.
 2. `test_api.py` não cobre os caminhos que dependem de variável de ambiente ausente (retorno 503) nem os fluxos completos de pagamento/webhook.
 3. Rate limiter Redis (`rate_limiter_redis.py`) nunca rodou contra um Redis real — o rate limiter em memória é o ativo por padrão.
 4. `Negociacoes/Excluir` (baixa de negatificação) só faz sentido se a empresa se tornar credor autorizado — não há fluxo de negócio real usando esse `client method` ainda.
 5. Sem migração de schema (Alembic ou similar) — mudança de schema hoje é manual, direto no `SCHEMA` de `database.py`. Considerar antes de qualquer alteração de tabela em produção com dado real.
 6. Sem CI configurado (GitHub Actions etc.) — testes rodam só manualmente. Recomendado configurar antes de aceitar PRs de mais de um dev.
-

Convenções de código a manter

- Nomes de função, variável e mensagem de erro em **português** — consistente em todo o codebase, não é inconsistência a corrigir
- Docstrings sempre explicam **por quê**, não só o quê — várias decisões de arquitetura só fazem sentido lendo o comentário (ex: por que `hash_deterministico_documento` existe separado do `hash de senha`)
- Todo módulo novo em `app/core/` deve continuar sem `import` de `httpx/fastapi/sqlite3` — se precisar de I/O, é `app/integrations/` ou fica em `main.py/database.py`
- Testes usam SQLite temporário real (`tempfile.NamedTemporaryFile`), nunca mock de banco — mantenha esse padrão em testes novos que toquem `database.py`